

Kolumnowe bazy danych cz. I

Eksploracja danych, podstawowe agregacje i pierwszy benchmark PostgreSQL i

ClickHouse

Imię i nazwisko: Jan Małek

Grupa: 4

Parametry komputera:

32GB RAM Procesor M1 Pro

Cel ćwiczenia

- sprawdzenie poprawności działania przygotowanego środowiska,
- zapoznanie się z tabelą events,
- przećwiczenie podstawowych zapytań analitycznych SQL,
- przygotowanie pierwszych prostych wskaźników KPI,
- porównanie wyników uzyskanych w PostgreSQL i ClickHouse,
- sformułowanie pierwszych wniosków, w jakich analizach baza kolumnowa może być szczególnie użyteczna.

Ważne

- Pracuj na tej samej tabeli events w obu bazach danych.
- W typowej konfiguracji na zajęciach wystarczy używać nazwy events.
- Jeżeli w Twoim kliencie SQL tabela nie jest widoczna w bieżącym kontekście, użyj pełnej nazwy: public.events w PostgreSQL albo ds_lab.events w ClickHouse.
- Tam, gdzie polecenie mówi o porównaniu, nie wystarczy samo uruchomienie zapytania - napisz krótko, czy wyniki są zgodne.
- Do każdego zadania dołącz: kod zapytania, wynik oraz wymagany komentarz.
- Nie oddawaj samych zrzutów wyników bez interpretacji.
- W kilku zadaniach możesz korzystać z zapytań startowych, ale część rozwiązań musisz samodzielnie rozbudować.
- Do benchmarku użyj tego samego klienta SQL dla obu baz danych.
- Być może na zajęciach zabraknie czasu na wykonanie wszystkich zadań. Pozostałe elementy należy dokończyć po zajęciach.

Opis kolumn tabeli events

- event_time - czas zdarzenia,
- event_type - typ zdarzenia, np. view, cart, purchase,
- user_id - identyfikator użytkownika,
- session_id - identyfikator sesji,
- country - kraj,
- device - urządzenie,
- product_id - identyfikator produktu,
- price - cena jednostkowa,
- quantity - liczba sztuk.

Sprawozdanie

Oddawane sprawozdanie powinno zawierać komplet rozwiązań wszystkich zadań: kod zapytań, wyniki oraz wymagane komentarze.

- Sprawozdanie oddaj jako plik PDF albo Markdown.
- Kod SQL formatuj jako bloki kodu.
- Wyniki zapytań przedstawiaj jako tabele albo zrzuty ekranu.
- Komentarze pisz pełnymi zdaniami.
- Termin oddania: Do końca dnia poprzedzającego kolejne zajęcia

Punktacja: razem 10 pkt.

0. Gotowość do zajęć - 0 pkt

Pokaż, że środowisko jest przygotowane do pracy.

- docker compose up -d uruchamia oba kontenery bez błędów,
- docker ps pokazuje kontenery postgres i clickhouse w statusie Up,
- w obu bazach dostępna jest tabela events,
- możliwe jest połączenie z obiema bazami z poziomu klienta SQL.

Uwaga: brak gotowości środowiska uniemożliwia wykonanie dalszych zadań.

1. Pierwsze poznanie tabeli events - 1 pkt

Zapoznaj się z tabelą events w obu bazach.

- sprawdź strukturę tabeli,
- wyświetl kilka przykładowych rekordów,
- policz liczbę wierszy,
- wyznacz minimalny i maksymalny czas zdarzenia,
- sprawdź, czy w kolumnach price i quantity występują wartości NULL, jeśli występują, wskaż ich liczbę.

Zapytania startowe:

```
SELECT * FROM events LIMIT 20;
```

Struktura tabeli i kilka przykładowych rekordów

	event_time	user_id	session_id	product_id	price	quantity	country	device	event_type
1	2025-05-05 03:56:41.000000	7297	6557	4507	126.22	1	DE	tablet	view
2	2025-04-16 21:35:32.000000	5698	154795	6913	20.73	1	IT	tablet	view
3	2025-04-19 23:35:29.000000	13032	187701	8929	212.66	1	ES	mobile	view
4	2025-02-01 00:03:58.000000	45754	110786	5575	142.55	1	US	mobile	add_to_cart
5	2025-01-19 18:43:51.000000	23527	98166	9892	135.94	1	SE	tablet	add_to_cart
6	2025-06-29 02:12:28.000000	24808	20657	9045	150.12	1	US	tablet	add_to_cart
7	2025-01-14 12:09:01.000000	3004	173347	3734	387.67	3	DE	desktop	view
8	2025-05-04 10:30:15.000000	23910	42639	6066	180.86	1	DE	tablet	view
9	2025-04-14 17:16:29.000000	47785	64176	2678	233.82	1	NO	mobile	view
10	2025-06-13 15:58:28.000000	3666	68044	526	403.5	1	DE	mobile	view
11	2025-04-21 03:15:02.000000	47050	82491	3484	329.44	1	SE	mobile	view
12	2025-02-17 21:22:02.000000	48824	147160	8831	135.86	1	NL	desktop	view
13	2025-01-27 20:40:21.000000	33393	129373	1490	379.11	3	FR	tablet	view
14	2025-01-13 08:03:38.000000	25217	100040	9764	498.88	1	NO	mobile	view
15	2025-01-23 05:50:59.000000	44677	140764	4372	385.46	1	ES	desktop	add_to_cart
16	2025-01-01 15:07:27.000000	47324	188660	4316	486.88	1	NO	mobile	view
17	2025-02-27 22:51:12.000000	41875	133081	9978	103.46	1	FR	tablet	view
18	2025-06-29 02:51:15.000000	34758	150	9814	165.46	1	US	desktop	view
19	2025-02-16 18:32:34.000000	37183	20646	1404	367.29	1	NO	mobile	add_to_cart
20	2025-04-03 06:58:35.000000	36032	43288	4343	266.2	1	IT	tablet	view

```
SELECT
count(*) AS n,
min(event_time) AS min_time,
max(event_time) AS max_time
FROM events;
```

Postgres

	n	min_time	max_time
1	1000000	2025-01-01 00:00:02.000000	2025-06-29 23:59:25.000000

ClickHouse

	n	min_time	max_time
1	1000000	2025-01-01 00:00:02	2025-06-29 23:59:25

Zakresy czasu są takie same, jedynie różnia się formatowanie. Postgres dodatkowo wyświetla czas z dokładnością do mikrosekund, ClickHouse wyświetla czas do pełnych sekund.

```
SELECT
count(*) AS all_rows,
count(price) AS non_null_price,
count(quantity) AS non_null_quantity,
count(*) - count(price) AS null_price_rows,
count(*) - count(quantity) AS null_quantity_rows
FROM events;
```

Postgres

	all_rows	non_null_price	non_null_quantity	null_price_rows	null_quantity_rows
1	1000000	1000000	1000000	0	0

ClickHouse

	all_rows	non_null_price	non_null_quantity	null_price_rows	null_quantity_rows
1	1000000	1000000	1000000	0	0

W badanych kolumnach nie ma w wynikach NULL, co widać na załączonych tabelach.

2. Profil danych zdarzeniowych - 1 pkt

Przygotuj podstawowy profil danych.

- sprawdź, jakie typy zdarzeń występują w danych i ile ich jest,
- sprawdź, jakie kraje występują w danych i ile mają zdarzeń,
- sprawdź, jakie urządzenia występują w danych i ile mają zdarzeń,
- dla każdego z tych trzech przekrojów wskaż kategorię dominującą.

Zapytanie startowe:

```
SELECT
event_type,
count(*) AS n
FROM events
GROUP BY event_type
ORDER BY n DESC;
```

	event_type	n
1	view	799261
2	add_to_cart	150406
3	purchase	50333

Występują 3 typy zdarzeń: "view": 799261, "add_to_cart": 150406 oraz "purchase": 50333.

Dwa pozostałe zapytania przygotuj samodzielnie na analogicznej zasadzie.

```
SELECT
country,
count(*) AS n
FROM events
GROUP BY country
ORDER BY n DESC;
```

	country	n
1	FR	100290
2	DE	100220
3	US	100193
4	NL	100061
5	ES	100022
6	NO	99972
7	PL	99954
8	GB	99893
9	IT	99734
10	SE	99661

Występuje 10 krajów, z których są użytkownicy. Każdy z nich posiada ok. 100tys. przypadków. Najliczniejsza jest FR.

```
SELECT
device,
count(*) AS n
FROM events
GROUP BY device
ORDER BY n DESC;
```

	device	n
1	mobile	333764
2	desktop	333355
3	tablet	332881

W obu serwerach ClickHouse i Postgres wyniki wychodzą te same.

W komentarzu: napisz 2-3 zdania podsumowania. Nie oddawaj tylko trzech niezależnych wyników bez wniosków.

3. Aktywność w czasie - 1 pkt

Sprawdź, jak zmienia się liczba zdarzeń w czasie.

- policz liczbę zdarzeń dla kolejnych dni,
- wskaż 5 dni o największej liczbie zdarzeń,
- wskaż 5 dni o najmniejszej liczbie zdarzeń,
- napisz, czy dane wyglądają na równomiernie rozłożone w czasie.

Możesz przygotować osobno: pełne zestawienie dzienne, 5 dni o największej liczbie zdarzeń oraz 5 dni o najmniejszej liczbie zdarzeń.

```
-- Postgres
SELECT
    DATE(event_time) AS event_date,
    COUNT(*) AS event_count
FROM events
GROUP BY event_date
ORDER BY event_date;

-- ClickHouse
SELECT
    toDate(event_time) AS event_date,
    COUNT(*) AS event_count
FROM events
GROUP BY event_date
ORDER BY event_date;
```

	event_date	event_count
1	2025-01-01	5499
2	2025-01-02	5470
3	2025-01-03	5617
4	2025-01-04	5468
5	2025-01-05	5573
6	2025-01-06	5512
7	2025-01-07	5443
8	2025-01-08	5577
9	2025-01-09	5559
10	2025-01-10	5587
11	2025-01-11	5342
12	2025-01-12	5594
13	2025-01-13	5479
14	2025-01-14	5684
15	2025-01-15	5569
16	2025-01-16	5641
17	2025-01-17	5567
18	2025-01-18	5592
19	2025-01-19	5457

```
-- Postgres
SELECT
    DATE(event_time) AS event_date,
    COUNT(*) AS event_count
FROM events
GROUP BY event_date
ORDER BY event_count DESC
LIMIT 5;

-- ClickHouse
SELECT
    toDate(event_time) AS event_date,
    COUNT(*) AS event_count
FROM events
GROUP BY event_date
ORDER BY event_count DESC
LIMIT 5;
```

	event_date	event_count
1	2025-02-21	5727
2	2025-02-04	5703
3	2025-04-23	5693
4	2025-02-24	5693
5	2025-02-20	5691

```
-- Postgres
SELECT
    DATE(event_time) AS event_date,
    COUNT(*) AS event_count
FROM events
GROUP BY event_date
ORDER BY event_count ASC
LIMIT 5;

-- ClickHouse
SELECT
    toDate(event_time) AS event_date,
    COUNT(*) AS event_count
FROM events
GROUP BY event_date
ORDER BY event_count ASC
LIMIT 5;
```

	event_date	event_count
1	2025-05-01	5315
2	2025-01-11	5342
3	2025-01-29	5382
4	2025-02-02	5397
5	2025-03-23	5407

Rozkład wygląda na stabilny różnice między, najmniejsza ilość, a największa ilość zdarzeń nie są zbyt duże (5727 do 5315). Nie ma widocznych, żadnych anomalii, odchylen. **W komentarzu:** napisz krótko, czy rozkład wygląda stabilnie, czy są widoczne dni odstające.

4. Podstawowe KPI sprzedażowe - 2 pkt

Przygotuj podstawowe KPI związane ze zdarzeniami zakupowymi.

- policz liczbę zdarzeń typu purchase,
- policz łączny przychód ze zdarzeń typu purchase,
- policz średnią wartość pojedynczego zakupu,
- policz średnią liczbę sztuk w pojedynczym zakupie,
- policz liczbę sesji, w których wystąpił co najmniej jeden zakup,
- policz udział sesji zakupowych w ogólnej liczbie sesji jako uproszczony współczynnik konwersji.

Zapytanie startowe:

```
SELECT
    count(*) AS purchases_cnt,
    sum(price * quantity) AS revenue,
    avg(price * quantity) AS avg_order_value,
    avg(quantity) AS avg_quantity
FROM events
WHERE event_type = 'purchase';
```

Drugą część zadania - dotyczącą sesji i konwersji - przygotuj samodzielnie.

```
WITH stats as (
    SELECT
        count(*) AS purchases_cnt,
        sum(price * quantity) AS revenue,
```

```
avg(price * quantity) AS avg_order_value,
avg(quantity) AS avg_quantity,
count(distinct session_id) AS buying_session
FROM events
WHERE event_type = 'purchase'
),
total as(
  select count(distinct session_id) as all_sessions
  from events
)
select
  s.*, t.all_sessions,
  100.0 * s.buying_session / t.all_sessions AS conversion_rate_pct
from stats s, total t
```

Postgres

	☐ purchases_cnt	☐ revenue	☐ avg_order_value	☐ avg_quantity	☐ buying_session	☐ all_sessions	☐ conversion_rate_pct
1	50333	17449170.359999955	346.67455466592406	1.3757177199849806	44601	198602	22.4574777696095709

ClickHouse

	☐ purchases_cnt	☐ revenue	☐ avg_order_value	☐ avg_quantity	☐ buying_session	☐ all_sessions	☐ conversion_rate_pct
1	50333	17449170.36	346.6745546659249	1.3757177199849807	44601	198602	22.45747776960957

Wyniki w obu bazach są ze sobą zgodne. Różnią się jedynie minimalnie w sposobie zaokrąglania liczb po przecinku. Udział sesji zakupowych lepiej opisuje konwersję, ponieważ, określa on jaki udział wizyt na sklepie zakończył się transakcją, niezależnie od ilości zobaczonych produktów. Jest to najbardziej interesująca dana w sprzedaży.

5. KPI w przekrojach biznesowych - 1 pkt

Policz przychód dla zdarzeń typu purchase w dwóch wybranych przekrojach:

- według kraju,
- według urządzenia,
- według dnia. Wybierz dwa z powyższych przekrojów. Dla każdego policz przychód, posortuj wynik malejąco, wskaż najwyższe wartości i krótko je zinterpretuj.

Zapytanie startowe:

```
SELECT
country,
sum(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase'
GROUP BY country
ORDER BY revenue DESC;
```

	☐ country	☐ revenue
1	FR	1805210.3200000061
2	IT	1779036.9600000042
3	PL	1774036.4399999988
4	NL	1753906.4700000002
5	SE	1749563.0099999993
6	GB	1748815.3699999973
7	NO	1733380.0400000021
8	ES	1707084.7800000001
9	US	1703552.5699999984
10	DE	1694584.3999999973

Drugie zapytanie przygotuj samodzielnie.

Największy przychód osiągnięto dla kraju FR, co może się również wiązać z największą ilością użytkowników z tego kraju.

```
SELECT
device,
```

```
sum(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase'
GROUP BY device
ORDER BY revenue DESC;
```

	device	revenue
1	tablet	5838853.9800000028
2	desktop	5835272.0900000027
3	mobile	5775044.2900000002

Największy przychód osiągnęły tablety, jednak generalnie są to równomiernie rozłożone statystyki.

```
SELECT
DATE(event_time),
sum(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase'
GROUP BY DATE(event_time)
ORDER BY revenue DESC;
```

	date	revenue
1	2025-05-26	118909.410000000002
2	2025-05-18	117428.909999999997
3	2025-02-15	116289.490000000005
4	2025-01-31	114908.339999999997
5	2025-03-31	113003.08
6	2025-04-30	111909.620000000007
7	2025-06-02	111583.070000000004
8	2025-04-24	111091.209999999998
9	2025-04-28	110957.4
10	2025-02-06	110455.500000000003
11	2025-05-24	108672.6
12	2025-02-10	108535.639999999996
13	2025-01-23	108065.509999999998
14	2025-06-16	107986.069999999999

Z tej tabeli można jedynie wywnioskować, że największe przychody w losowych dniach osiągane są w pierwszej połowie roku.

6. Użytkownicy o najwyższym przychodzie - 1 pkt

Znajdź użytkowników o najwyższym łącznym przychodzie z zakupów.

Wynik powinien zawierać: user_id, liczbę wszystkich zdarzeń użytkownika, liczbę zdarzeń typu purchase, łączny przychód użytkownika ze zdarzeń typu purchase. Pokaż co najmniej 20 rekordów o najwyższym przychodzie.

Ważne: Przychód licz tylko dla zdarzeń typu purchase.

Wskazówka: W tym zadaniu przydatne może być warunkowe zliczanie i warunkowe sumowanie.

Zapytanie startowe do uzupełnienia:

```
SELECT
user_id,
count(*) AS all_events,
... AS purchase_events,
... AS revenue
FROM events
GROUP BY user_id
ORDER BY revenue DESC
LIMIT 20;
```

W komentarzu napisz:

- czy użytkownik z najwyższym przychodem ma też największą liczbę wszystkich zdarzeń,
- czy duża aktywność użytkownika zawsze oznacza wysoki przychód,
- jakie wnioski można wyciągnąć z porównania liczby zdarzeń i przychodu.

```
SELECT
user_id,
count(*) AS all_events,
count(case when event_type='purchase' then 1 end) AS purchase_events,
sum(case when event_type='purchase' then price*quantity else 0 end) AS revenue
FROM events
GROUP BY user_id
ORDER BY revenue DESC
LIMIT 20;
```

	user_id	all_events	purchase_events	revenue
1	21194	25	4	5037.99
2	11919	23	3	4859.679999999999
3	18860	27	3	4639.92
4	21978	26	3	4604.17
5	39652	27	3	4584.65
6	35735	29	5	4527.26
7	32592	24	4	4526.1
8	48742	21	5	4460.98
9	4935	26	4	4360.1200000000001
10	9978	24	3	4301.87
11	33799	22	2	4204.8
12	29614	26	7	4192.2
13	36059	18	4	4160.8200000000001
14	7178	25	4	4119.37
15	37756	25	5	4078.35
16	23725	23	3	4006.1099999999997
17	7009	22	3	4001.56
18	3112	24	4	3980.3599999999997
19	47934	27	4	3915.71
20	48299	32	3	3866.2

Użytkownik z największym przychodem nie ma największej liczby zdarzeń. Nie zawsze duża aktywność użytkownika oznacza duży przychód, np użytkownik w wierszu 20, ma wszystkich wejść 32, a jednak nie jest na przodzie tabeli i jego przychód jest o ponad 1000 niższy od 1 miesiąca. Dodatkowo różnica między wszystkimi eventami, a tymi zakupowymi jest duża. Można rozróżnić użytkowników, którzy nie potrzebują dużej ilości wejść, od tych którzy dłużej zastanawiają się nad zakupem.

7. Benchmark zapytań w PostgreSQL i ClickHouse – 3 pkt

W tym zadaniu wykonasz prosty benchmark zapytań w PostgreSQL i ClickHouse, aby porównać działanie obu systemów dla zapytań prostszych i bardziej złożonych agregacyjnie. Szczegóły zostały opisane w częściach A, B i C.

Sposób pomiaru

Dla każdego benchmarkowanego zapytania:

- uruchom zapytanie w PostgreSQL i w ClickHouse,
- wykonaj każde zapytanie 4 razy w każdej bazie,
- pierwszy pomiar pomiń jako rozgrzewkowy,
- zapisz czasy z trzech kolejnych uruchomień,
- podaj średni czas wykonania dla każdej bazy. Czas odczytaj z klienta SQL, którego używasz na zajęciach.

Uwaga: nie benchmarkuj zapytań zwracających bardzo duży wynik bez LIMIT, ponieważ czas prezentacji danych w kliencie SQL może zaburzać porównanie.

Prezentacja wyników

Wyniki benchmarku przedstaw w zwartej tabeli. Tabela powinna zawierać co najmniej: nazwę zapytania, PostgreSQL - pomiar 1, pomiar 2, pomiar 3, średnia, oraz ClickHouse - pomiar 1, pomiar 2, pomiar 3, średnia.

Komentarz końcowy

Na końcu napisz 3-5 zdań komentarza, w których odniesiesz się do następujących kwestii:

- czy wyniki liczbowe zapytań były zgodne w obu bazach?
- czy różnice czasu wykonania były małe czy wyraźne?
- przy którym typie zapytania różnica była największa?
- czy po tym ćwiczeniu można postawić wstępny wniosek, że bardziej złożone agregacje są naturalnym zastosowaniem systemu analitycznego takiego jak ClickHouse?

Uwaga: nie oczekujemy jeszcze pełnego benchmarku technicznego. Celem zadania jest pierwsze praktyczne porównanie działania prostszych i bardziej złożonych zapytań agregujących w dwóch systemach.

Część A. Dwa zapytania z wcześniejszych zadań

Wybierz dwa zapytania z wcześniejszych zadań tego laboratorium i wykonaj je w obu bazach danych. Dla każdego zapytania pokaż wynik, napisz, czy wynik liczbowy jest zgodny w PostgreSQL i ClickHouse, oraz zapisz czas wykonania odczytany z klienta SQL.

Wybierz zapytania o różnym poziomie trudności, np. jedno prostsze zapytanie i jedno średnio złożone zapytanie.

```
SELECT
count(*) AS n,
min(event_time) AS min_time,
max(event_time) AS max_time
FROM events;
```

Postgres

	n	min_time	max_time
1	1000000	2025-01-01 00:00:02.000000	2025-06-29 23:59:25.000000

461 ms (execution: 136 ms, fetching: 325 ms)
398 ms (execution: 83 ms, fetching: 315 ms)
389 ms (execution: 72 ms, fetching: 317 ms)

ClickHouse

	n	min_time	max_time
1	1000000	2025-01-01 00:00:02	2025-06-29 23:59:25

330 ms (execution: 9 ms, fetching: 321 ms)
337 ms (execution: 11 ms, fetching: 326 ms)
336 ms (execution: 10 ms, fetching: 326 ms)

```
WITH stats as (
  SELECT
    count(*) AS purchases_cnt,
    sum(price * quantity) AS revenue,
    avg(price * quantity) AS avg_order_value,
    avg(quantity) AS avg_quantity,
    count(distinct session_id) AS buying_session
  FROM events
  WHERE event_type = 'purchase'
),
total as(
  select count(distinct session_id) as all_sessions
  from events
)
select
  s.*, t.all_sessions,
  100.0 * s.buying_session / t.all_sessions AS conversion_rate_pct
from stats s, total t
```

Postgres

	purchases_cnt	revenue	avg_order_value	avg_quantity	buying_session	all_sessions	conversion_rate_pct
1	50333	17449170.359999955	346.67455466592406	1.3757177199849006	44601	198602	22.4574777696095709

```
928 ms (execution: 591 ms, fetching: 337 ms)
745 ms (execution: 418 ms, fetching: 327 ms)
730 ms (execution: 407 ms, fetching: 323 ms)
```

ClickHouse

	purchases_cnt	revenue	avg_order_value	avg_quantity	buying_session	all_sessions	conversion_rate_pct
1	50333	17449170.36	346.6745546659249	1.3757177199849007	44601	198602	22.45747776960957

```
378 ms (execution: 37 ms, fetching: 341 ms)
351 ms (execution: 31 ms, fetching: 320 ms)
458 ms (execution: 139 ms, fetching: 319 ms)
```

Przy prostym zapytaniu widac lekka przewage ClickHouse, która staje się dużo bardziej widoczna przy bardziej złożonym zapytaniu. Czas mimo tego ze ms, jest dwukrotnie mniejszy pod Postgresa. Bardziej złożone zapytania, są dużo lepsze dla systemów kolumnowych, które wczytują tylko potrzebne kolumny, w porównaniu do wierszowych takich jak Postgres.

Część B. Zapytanie benchmarkowe podane przez prowadzącego

Wykonaj w obu bazach poniższe zapytanie agregujące.

PostgreSQL:

```
SELECT
DATE(event_time) AS day,
country,
device,
event_type,
count(*) AS events_cnt,
count(DISTINCT user_id) AS users_cnt,
count(DISTINCT session_id) AS sessions_cnt,
sum(CASE
WHEN event_type = 'purchase' THEN price * quantity
ELSE 0
END) AS revenue
FROM events
GROUP BY
DATE(event_time),
country,
device,
event_type
ORDER BY revenue DESC
LIMIT 20;
```

ClickHouse:

```
SELECT
toDate(event_time) AS day,
country,
device,
event_type,
count() AS events_cnt,
count(DISTINCT user_id) AS users_cnt,
count(DISTINCT session_id) AS sessions_cnt,
sum(CASE
WHEN event_type = 'purchase' THEN price * quantity
ELSE 0
END) AS revenue
FROM events
GROUP BY
day,
country,
device,
event_type
ORDER BY revenue DESC
LIMIT 20;
```

	day	country	device	event_type	events_cnt	users_cnt	sessions_cnt	revenue
1	2025-06-13	US	mobile	purchase	14	14	14	18827.559999999998
2	2025-06-21	US	tablet	purchase	21	21	21	9945.939999999997
3	2025-01-23	IT	tablet	purchase	19	19	19	9584.159999999998
4	2025-05-15	IT	desktop	purchase	15	15	15	9468.63
5	2025-05-30	DE	desktop	purchase	18	18	18	9445.289999999999
6	2025-03-04	US	tablet	purchase	18	18	18	9403.07
7	2025-01-04	GB	tablet	purchase	20	20	20	9342.119999999999
8	2025-02-04	FR	tablet	purchase	21	21	21	9139.390000000003
9	2025-02-24	IT	desktop	purchase	16	16	16	9022.180000000002
10	2025-04-19	PL	mobile	purchase	15	15	15	8954.64
11	2025-05-25	DE	mobile	purchase	15	15	15	8909.98
12	2025-06-10	DE	desktop	purchase	21	21	21	8884.72
13	2025-03-24	IT	desktop	purchase	17	17	17	8801.78
14	2025-03-20	IT	desktop	purchase	11	11	11	8796.71
15	2025-05-10	IT	tablet	purchase	15	15	15	8755.74
16	2025-04-19	ES	mobile	purchase	13	13	13	8738.15
17	2025-06-14	PL	tablet	purchase	14	14	14	8689.470000000001
18	2025-04-26	US	tablet	purchase	14	14	14	8663.769999999999
19	2025-03-25	SE	mobile	purchase	15	15	15	8570.769999999999
20	2025-04-18	FR	mobile	purchase	13	13	13	8570.720000000001

2 s 482 ms (execution: 2 s 98 ms, fetching: 384 ms)
2 s 348 ms (execution: 2 s 25 ms, fetching: 323 ms)
2 s 292 ms (execution: 1 s 972 ms, fetching: 320 ms)

	day	country	device	event_type	events_cnt	users_cnt	sessions_cnt	revenue
1	2025-06-13	US	mobile	purchase	14	14	14	18827.56
2	2025-06-21	US	tablet	purchase	21	21	21	9945.939999999999
3	2025-01-23	IT	tablet	purchase	19	19	19	9584.16
4	2025-05-15	IT	desktop	purchase	15	15	15	9468.630000000001
5	2025-05-30	DE	desktop	purchase	18	18	18	9445.29
6	2025-03-04	US	tablet	purchase	18	18	18	9403.070000000002
7	2025-01-04	GB	tablet	purchase	20	20	20	9342.12
8	2025-02-04	FR	tablet	purchase	21	21	21	9139.39
9	2025-02-24	IT	desktop	purchase	16	16	16	9022.180000000002
10	2025-04-19	PL	mobile	purchase	15	15	15	8954.640000000001
11	2025-05-25	DE	mobile	purchase	15	15	15	8909.98
12	2025-06-10	DE	desktop	purchase	21	21	21	8884.72
13	2025-03-24	IT	desktop	purchase	17	17	17	8801.78
14	2025-03-20	IT	desktop	purchase	11	11	11	8796.710000000001
15	2025-05-10	IT	tablet	purchase	15	15	15	8755.74
16	2025-04-19	ES	mobile	purchase	13	13	13	8738.15
17	2025-06-14	PL	tablet	purchase	14	14	14	8689.470000000001
18	2025-04-26	US	tablet	purchase	14	14	14	8663.77
19	2025-03-25	SE	mobile	purchase	15	15	15	8570.77
20	2025-04-18	FR	mobile	purchase	13	13	13	8570.720000000001

405 ms (execution: 67 ms, fetching: 338 ms)
384 ms (execution: 57 ms, fetching: 327 ms)
381 ms (execution: 58 ms, fetching: 323 ms)

Dla tego zapytania pokaż wynik z obu baz, napisz, czy wyniki są zgodne, oraz zapisz czas wykonania w PostgreSQL i ClickHouse.

Wyniki tak jak wcześniej, różnią się w niektórych miejscach zaokrągleniem przecinków. Czasy wykonania zdecydowanie pokazują przewagę ClickHouse w porównaniu do Postgresa, który jest kilka razy wolniejszy.

Część C. Własne analogiczne zapytanie

Na podstawie zapytania z części B przygotuj jedno własne zapytanie analogiczne, które będzie oparte na tym samym schemacie, ale zmodyfikowane w co najmniej jednym lub dwóch elementach.

Możesz skorzystać z poniższych modyfikacji:

- ograniczyć dane tylko do zdarzeń purchase,
- usunąć jeden wymiar grupowania,
- dodać filtr dla wybranego kraju,
- dodać filtr dla wybranego urządzenia,
- zmienić zestaw liczonych miar,
- ograniczyć analizę do wybranego przedziału czasu.

Dodano ograniczenie tylko dla "purchase" i dla FR

```
SELECT
    DATE(event_time) AS day,
    country,
    device,
    count(*) AS purchases_cnt,
    count(DISTINCT user_id) AS unique_buyers,
    count(DISTINCT session_id) AS buying_sessions,
    sum(price * quantity) AS daily_revenue
FROM events
WHERE event_type = 'purchase'
    AND country = 'FR'
```

```
GROUP BY
    day,
    country,
    device
ORDER BY daily_revenue DESC
LIMIT 20;
```

```
SELECT
    toDate(event_time) AS day,
    country,
    device,
    count() AS purchases_cnt,
    uniq(user_id) AS unique_buyers,
    uniq(session_id) AS buying_sessions,
    sum(price * quantity) AS daily_revenue
FROM events
WHERE event_type = 'purchase'
    AND country = 'FR'
GROUP BY
    day,
    country,
    device
ORDER BY daily_revenue DESC
LIMIT 20;
```

Postgres

	day	country	device	purchases_cnt	unique_buyers	buying_sessions	daily_revenue
1	2025-02-04	FR	tablet	21	21	21	9139.390000000003
2	2025-04-18	FR	mobile	13	13	13	8570.720000000001
3	2025-02-18	FR	mobile	16	16	16	8520.26
4	2025-01-09	FR	mobile	16	16	16	8297.74
5	2025-04-30	FR	mobile	13	13	13	8294.35
6	2025-01-22	FR	desktop	15	15	15	8229.66
7	2025-06-14	FR	mobile	17	17	17	7668.690000000005
8	2025-03-29	FR	desktop	18	18	18	7426.1500000000015
9	2025-06-22	FR	tablet	12	12	12	7112.439999999999
10	2025-03-03	FR	tablet	20	20	20	6968.81
11	2025-05-27	FR	tablet	11	11	11	6944.2
12	2025-03-27	FR	desktop	15	15	15	6918.150000000001
13	2025-05-02	FR	tablet	9	9	9	6891.29
14	2025-02-17	FR	desktop	12	12	12	6799.25
15	2025-05-13	FR	tablet	15	15	15	6700.59
16	2025-05-15	FR	tablet	14	14	14	6655.55
17	2025-05-26	FR	tablet	15	15	15	6640.92
18	2025-03-31	FR	mobile	13	13	13	6602.000000000001
19	2025-06-27	FR	mobile	13	13	13	6561.450000000001
20	2025-02-04	FR	desktop	8	8	8	6560.68

458 ms (execution: 117 ms, fetching: 341 ms)
420 ms (execution: 98 ms, fetching: 322 ms)
405 ms (execution: 86 ms, fetching: 319 ms)

ClickHouse

	day	country	device	purchases_cnt	unique_buyers	buying_sessions	daily_revenue
1	2025-02-04	FR	tablet	21	21	21	9139.39
2	2025-04-18	FR	mobile	13	13	13	8570.720000000001
3	2025-02-18	FR	mobile	16	16	16	8520.260000000002
4	2025-01-09	FR	mobile	16	16	16	8297.740000000002
5	2025-04-30	FR	mobile	13	13	13	8294.35
6	2025-01-22	FR	desktop	15	15	15	8229.659999999998
7	2025-06-14	FR	mobile	17	17	17	7668.690000000005
8	2025-03-29	FR	desktop	18	18	18	7426.150000000001
9	2025-06-22	FR	tablet	12	12	12	7112.439999999999
10	2025-03-03	FR	tablet	20	20	20	6968.8099999999995
11	2025-05-27	FR	tablet	11	11	11	6944.2
12	2025-03-27	FR	desktop	15	15	15	6918.15
13	2025-05-02	FR	tablet	9	9	9	6891.289999999998
14	2025-02-17	FR	desktop	12	12	12	6799.25
15	2025-05-13	FR	tablet	15	15	15	6700.59
16	2025-05-15	FR	tablet	14	14	14	6655.55
17	2025-05-26	FR	tablet	15	15	15	6640.92
18	2025-03-31	FR	mobile	13	13	13	6602
19	2025-06-27	FR	mobile	13	13	13	6561.449999999999
20	2025-02-04	FR	desktop	8	8	8	6560.68

381 ms (execution: 32 ms, fetching: 349 ms)
347 ms (execution: 30 ms, fetching: 317 ms)
348 ms (execution: 28 ms, fetching: 320 ms)

Wyniki obu baz są zgodne, czasowo ClickHouse nadal jest lepszy, jednak przy bardziej ograniczonym zbiorze te różnice czasowe nie są takie duże.

Uwaga końcowa dla studentów

Oceniane będą:

- poprawność logiczna zapytania,
- zgodność wyników między bazami tam, gdzie jest to wymagane,
- umiejętność interpretacji wyniku,
- czytelność i sensowność kodu. Samo uruchomienie gotowego zapytania bez zrozumienia i bez komentarza nie będzie traktowane jako pełne rozwiązanie.